

RailRoute — COA Task 2 Documentation

64-bit Assembly-Based Application Processing (CO2)

File: src/assembly/distance_calc.asm

1. Register Table

Register	Used for	Width	Notes
RAX	Op 1: running distance accumulator. Op 2: Route A distance / printf return value (ignored)	64-bit	Caller-saved; reloaded fresh at the start of each operation
RBX	Op 2: Route B distance	64-bit	Callee-saved by the Win64 ABI — safe to hold across the printf calls without reloading
RCX	1st integer argument to printf (format-string pointer)	64-bit (pointer)	Required by the Microsoft x64 calling convention
RDX	2nd integer argument to printf (the %lld value): result / difference	64-bit	Reused per call — not persistent state
RSP/RBP	Stack frame + 32-byte “shadow space” required by Win64 ABI before every call	64-bit	Standard prologue/epilogue

2. Instruction & Control-Flow Explanation

Operation 1 — Aggregation

```
mov rax, [seg1_nagpur_pune]
add rax, [seg2_pune_nashik]
add rax, [seg3_nashik_mumbai]
```

MOV performs a 64-bit memory→register load. ADD performs 64-bit register+memory addition, updating RAX in place. No overflow risk here (result ≈ 1.1 million, far under the 64-bit ceiling of $\sim 1.8 \times 10^{19}$) —

this is the honestly-stated limitation: the *values* don't need 64 bits, but every instruction and register genuinely operates at 64-bit width as CO2 requires.

Operation 2 — Comparison

```
mov rax, [routeA_distance]
mov rbx, [routeB_distance]
cmp rax, rbx
je routes_equal
jl route_a_shorter
jg route_b_shorter
```

CMP internally performs RAX – RBX and discards the result, only setting FLAGS: **ZF** (zero flag, set if equal), **SF/OF** (sign/overflow flags, together determine “less than” for signed 64-bit values). JE jumps if ZF=1. JL jumps if SF≠OF (RAX < RBX, signed). JG jumps if ZF=0 and SF=OF (RAX > RBX, signed). Each branch then computes the positive difference with SUB and prints via printf.

This is a genuine full 64-bit comparison — CMP RAX, RBX compares the entire 64-bit register in one instruction, not high/low halves, because both operands already fit natively in 64-bit registers (no manual high/low split was needed since we didn't use 32-bit halves anywhere).

3. RailRoute-to-COA Mapping

Assembly element	RailRoute meaning
seg1_nagpur_pune, seg2_pune_nashik, seg3_nashik_mumbai	Real segment distances from RouteManager (Nagpur-Pune 720km, Pune-Nashik 210km, Nashik-Mumbai 165km)
Operation 1 result (1,095,000 m)	Total distance an Operator would see for a 3-leg journey — an analytics metric
routeA_distance / routeB_distance	The two paths a Passenger could take from Pune to Mumbai — direct (192km) vs. via Nashik (375km)
Operation 2 branch outcome	Backing logic for the PRD's “Intelligent Routing: Fastest vs Shortest route” feature

4. Test Plan & Results

Test	Route A	Route B	Expected	Actual (Linux-ABI equivalent run)
1: A < B	192,000	375,000	A shorter, diff 183,000	✓ matched
2: A > B	400,000	375,000	B shorter, diff 25,000	✓ matched
3: A = B	375,000	375,000	Equal	✓ matched
Op 1	720,000+210,000+165,000	—	1,095,000	✓ matched

sum

Status: logic validated via an ABI-equivalent Linux build (NASM elf64 + gcc) that I assembled, linked, and ran myself. The actual Windows .exe (NASM win64 + MinGW-w64) assembles and links cleanly into a valid PE32+ executable, but has **not yet been executed** — that confirmation is still outstanding from your side. Please don't mark this "final validated" in your report until you've run distance_calc.exe yourself and it matches the table above.

5. Viva Questions & Answers

1. **Why x86-64 / NASM?** It's the native instruction set of the lab PCs and directly gives real 64-bit registers, unlike a 16/32-bit simulator.
 2. **Why is this RailRoute-related?** The operands are real segment distances from RouteManager; Operation 2 mirrors the app's actual shortest-vs-fastest routing decision.
 3. **Why 64-bit data here specifically?** CO2 mandates it; also justified because operator-level cumulative totals (across many trips) could realistically grow past 32-bit range even though this single example doesn't.
 4. **Which registers, and why?** RAX/RBX for the two operands (RBX chosen because it's callee-saved and survives across printf), RCX/RDX for the Win64 calling convention's argument slots.
 5. **How does 64-bit addition work?** ADD adds the full 64-bit source into the full 64-bit destination register, propagating carry across all 64 bits in hardware.
 6. **How does CMP work?** It computes RAX-RBX internally (without storing the result) purely to set FLAGS.
 7. **Which flags, and why these jumps?** ZF for equality; SF vs OF together determine signed "less than" — JL/JG use that combination, not JB/JA (which are for *unsigned* comparisons — distances are never negative here, but treating them as signed 64-bit is the ABI-standard approach and still correct for these positive values).
 8. **How do you decide which route is shorter?** Whichever register the CMP+conditional-jump sequence identifies via the FLAGS register, not by pre-computing in code.
 9. **What happens when equal?** Falls through the JL/JG conditions, JE fires, and a distinct "Equal" branch executes — no difference is computed.
 10. **How did you validate the output?** Independent manual addition/subtraction of the same numbers, plus a working ABI-equivalent Linux run of the identical logic.
 11. **32-bit vs 64-bit difference here?** A 32-bit register (EAX) can only hold values up to ~4.29 billion; RAX's range is far larger — irrelevant for these specific numbers, but required because the CO explicitly asks for 64-bit processing.
 12. **Limitations?** Values used don't actually need the full 64-bit range; no overflow handling is implemented since it isn't reachable with these operands; distances are hardcoded, not read from the live C++ program at runtime.
 13. **How could this integrate into the full app?** The Java/C++ layers could shell out to distance_calc.exe and parse its output, or the distance values could be passed in via command-line arguments instead of being hardcoded — documented as a future extension, not implemented here.
-

6. AI-Use Record (fill in what you actually did)

Field	Content
AI tool used	Claude (Anthropic)
Purpose	Drafting and validating the COA Task 2 NASM implementation against RailRoute's existing route data
Prompt(s) used	<i>(paste your actual prompts from this conversation)</i>
Output summary	distance_calc.asm — 64-bit route aggregation + 64-bit route comparison, Win64 ABI
Problems/limitations found by AI itself	Initial draft used the wrong (Linux) calling convention; corrected before delivery. Values don't inherently require 64-bit range.
Corrections YOU actually made	<i>(fill in only if you changed anything yourself)</i>
Tests YOU performed	<i>(fill in once you've run distance_calc.exe on your machine)</i>
Final validation	<i>(pending your own execution)</i>